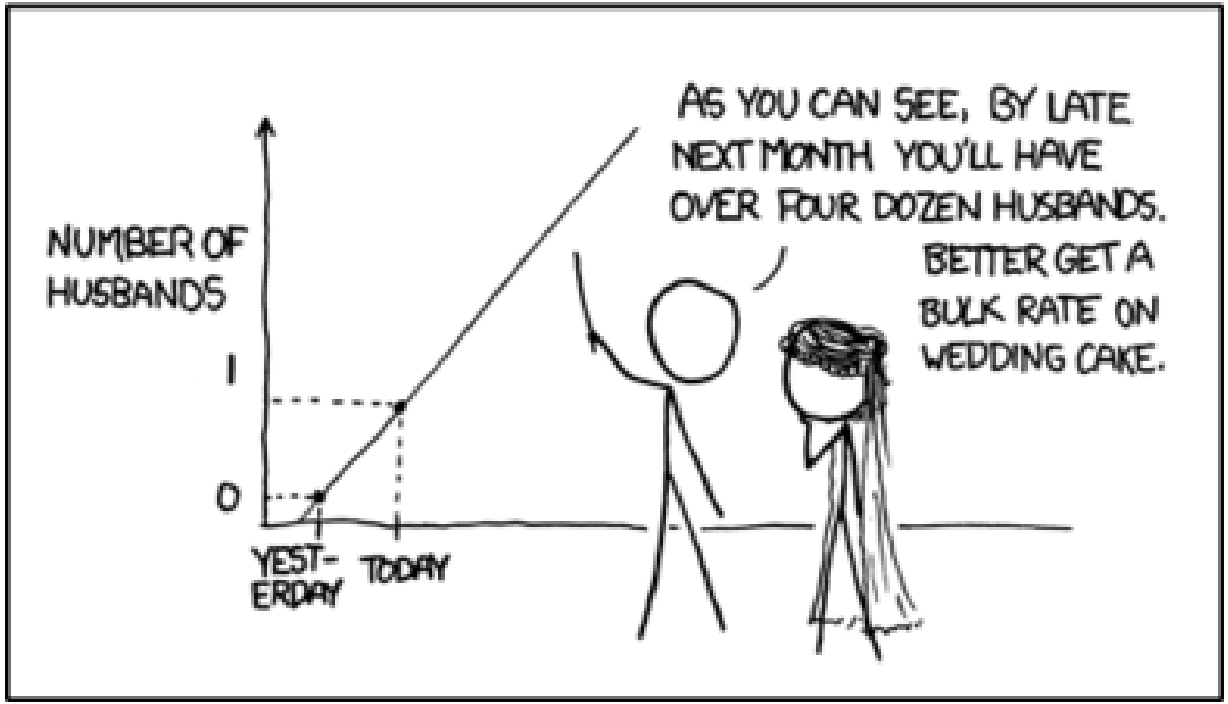
Regression

Machine learning

Doc. Ing. Radim Kolar, Ph.D.
Department of Biomedical Engineering,
FEEC, Brno University of Technology

Outline

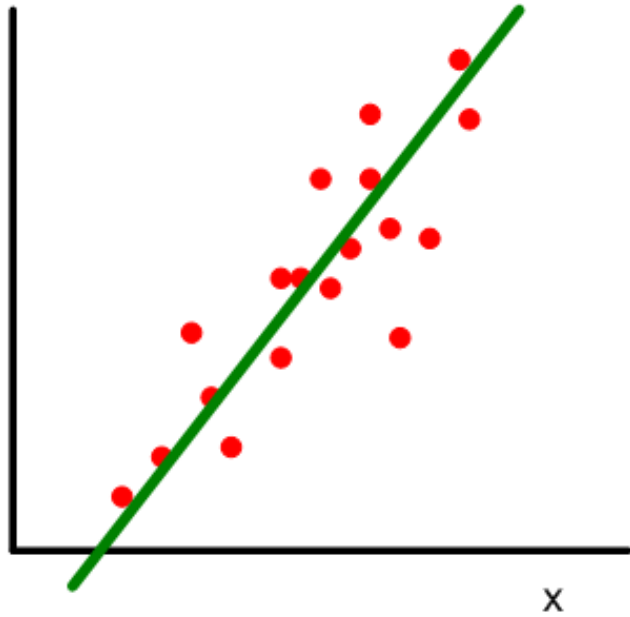
1. Introduction
2. Linear regression
3. Regularization - Ridge and Lasso
4. Non-linear regression
5. Logistic regression



https://mlcourse.ai/

Introduction

Linear Regression is a **supervised** machine learning algorithm where the predicted output is continuous and has a constant slope. It's used to predict values within a continuous range, (e.g. price) rather than trying to classify them into categories (e.g. cat, dog). There are two main types:



- **univariate** regression: $y = k.x + q$
- **multivariate** regression, e.g. $y = w_1x_1 + w_2x_2 + w_3x_3 + b$

The regression is a part of data analysis, where most univariate analysis emphasizes description of our data, while multivariate methods emphasize hypothesis testing and explanation (including feature analysis).

Linear regression

Linear regression

Data regression problems comes in the form of a (training) dataset of P observation pairs:

$$\{[\mathbf{x'}_1, y_1], [\mathbf{x'}_2, y_2], \ldots [\mathbf{x'}_p, y_p] \}$$

where $\mathbf{x'}_i$ denotes the input variables and y_i denotes the output. Both variables are generally continuous.

The simplest case is, if input variable is a scalar - in that case we are solving 2-dimensional problem, i.e. fitting a line in 2D space through scattered data. In general, however, each input $\mathbf{x'_i}$ may be a column vector of length n:

$$\mathbf{x'_i} = \begin{bmatrix} x_{1,i} \\ x_{2,i} \\ . \\ . \\ x_{n,i} \end{bmatrix}$$

n is a dimension of input vector, number of features

p is a number of datapoints

Linear regression

In this case the linear regression problem is fitting a hyperplane to a scatter of points in n+1 dimensional space.

In the case of scalar input, fitting a line to the data requires that we determine a slope *w* and a bias *b* of the regression line so that the approximate linear relationship holds between input/output data:

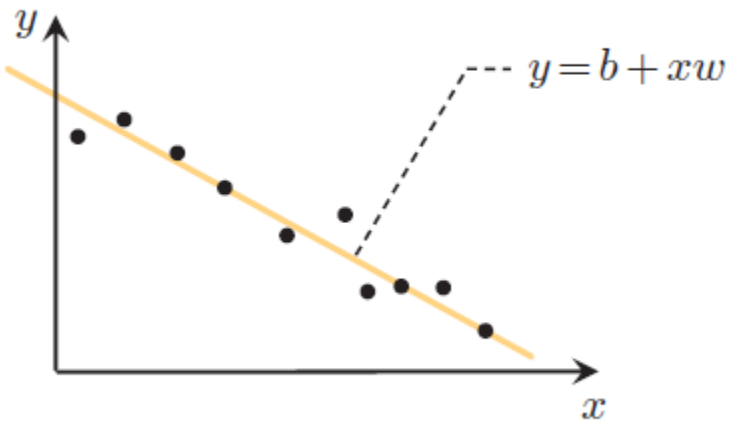
$$y_i \approx x_i w + b, \quad i = 1, 2, \ldots p$$

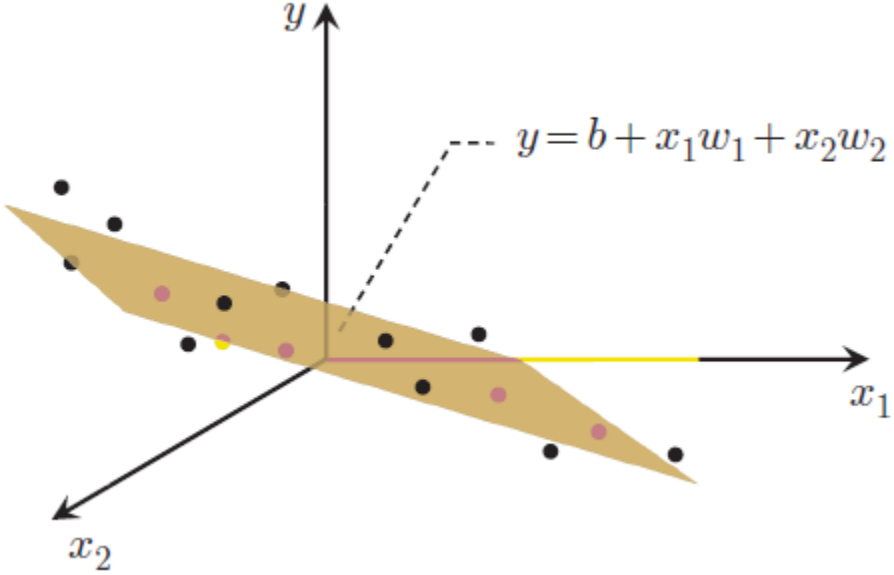
Linear regression

More generally, when the input dimension is n ≥ 1, we have a bias and n associated weights:

$$\mathbf{w}' = \begin{bmatrix} w_1 \\ w_2 \\ \cdot \\ \cdot \\ w_n \end{bmatrix}$$

The weights determine the normal vector of the hyperplane.

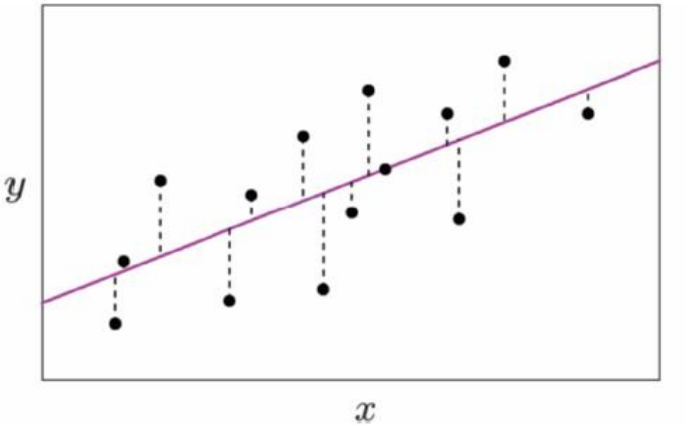




Linear regression

To find the parameters of the hyperplane which best fits a regression dataset, it is common practice to first form the **Least Squares cost function**.

For a given set of parameters (b,**w’**) this cost function computes the total squared error between the associated hyperplane and the data, giving a good measure of how well the particular linear model fits the dataset. Naturally then the best fitting hyperplane is the one whose parameters minimize this error.



Linear regression

To form the desired cost we simply square the difference (or error) between the linear model ($b + \mathbf{x}_i^T\mathbf{w}'$) and the corresponding output y_i over the entire dataset. This gives the Least Squares cost function:

$$g(b, \mathbf{w}') = \sum_{i=1}^{p} (b + \mathbf{x}_\mathbf{i}'^{\mathbf{T}}\mathbf{w}' - y_i)^2$$

We of course want to find a parameter pair *(b,**w'**)* that provides a small value for *g(b,**w'**)* since the larger this value the larger the squared error between the corresponding linear model and the data, and hence the poorer we represent the given data. Therefore we aim to minimize *g* over the bias *b* and weight vector ***w'*** in order to recover the best pair *(b,**w'**)*, which is written formally:

$$\underset{b,\mathbf{w}'}{minimize} \quad \sum_{i=1}^{p} (b + \mathbf{x}'^{\mathbf{T}}\mathbf{w}' - y_i)^2$$

Linear regression

Now that we have a minimization problem to solve. To perform calculations, it will first be convenient to use the following more compact notation (just to simplify it):

$$\mathbf{x_i} = \begin{bmatrix} 1 \\ \mathbf{x_i'} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} b \\ \mathbf{w'} \end{bmatrix}$$

These “new” vectors are called as **augmented** vectors.

With this notation we can rewrite the cost function in terms of the single vector **w** and introduce matrix notation:

$$g(\mathbf{w}) = \sum_{i=1}^{p} (\mathbf{x_i^T w} - y_i)^2 = ||\mathbf{Xw} - \mathbf{y}||^2$$

The matrix of inputs is:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{p1} & \cdots & x_{pn} \end{bmatrix}$$

And the vector of outputs:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_p \end{bmatrix}$$

Linear regression

The derivative of this cost function with respect to **w** is simple:

$$\nabla g(\mathbf{w}) = \sum_{i=1}^{p} 2(\mathbf{w}^T \mathbf{x_i} - y_i)\mathbf{x_i} = 2\mathbf{X^T}(\mathbf{Xw} - \mathbf{y})$$

And we set this gradient to zero.

Linear regression

Here, we can set the gradient to zero:

$$2\mathbf{X}^{\mathbf{T}}(\mathbf{X}\mathbf{w} - \mathbf{y}) = 0$$

We are looking for **w**:

$$\mathbf{X}^{\mathbf{T}}\mathbf{X}\mathbf{w} = \mathbf{X}^{\mathbf{T}}\mathbf{y}$$

And finally:

$$\mathbf{w} = (\mathbf{X}^{\mathbf{T}}\mathbf{X})^{-1}\mathbf{X}^{\mathbf{T}}\mathbf{y} = \mathbf{X}^{\dagger}\mathbf{y}$$

This is a direct solution, which needs the inverse matrix of $\mathbf{X^TX}$, which is square and often nonsingular (one that has inverse matrix). Term $\mathbf{(X^TX)^{-1}X^T}$ is also known as pseudo-inverse.

Linear regression

The last equation is a practical solution - we have to construct the matrix **X** and vector **y** and we are able to compute the **w**.

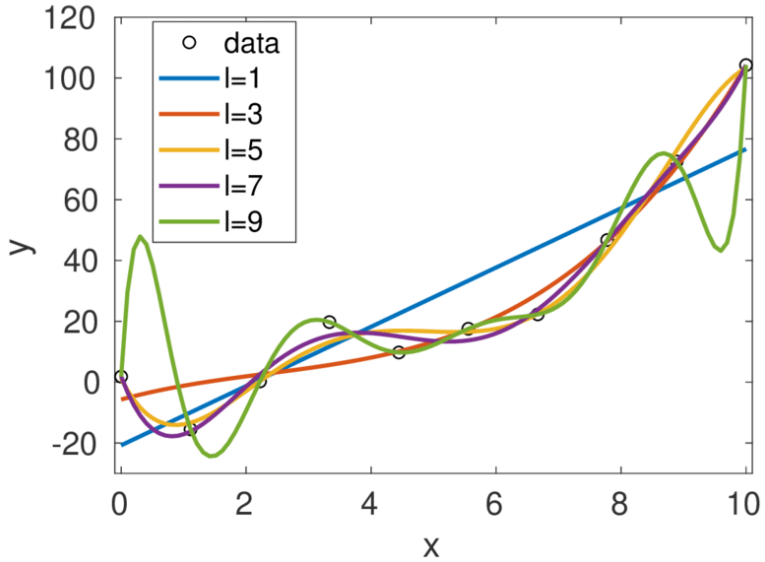
Univariate regression is simple to compute and the interpretation and visualization is straightforward.

In multivariate case, the analysis can become more tricky as the number of variables grows. There are some other techniques/modifications, which are more convenient for multivariate data analysis.

Polynomial regression

We model the data by m-th degree polynomial in x, i.e.:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_m x_i^m$$



Polynomial regression

This model can be expressed in matrix form in terms of a design matrix **X**, a response vector **y**, a parameter vector $\boldsymbol{\beta}$. The i-th row of **X** and **y** will contain the x_i and y_i value for the i-th data sample. Then the model can be written as a system of linear equations:

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^m \\ 1 & x_2 & x_2^2 & \dots & x_2^m \\ 1 & x_3 & x_3^2 & \dots & x_3^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^m \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \vdots \\ \beta_m \end{bmatrix},$$

Estimation of regression coefficient is formally the same as for linear regression. Only the **X** matrix is computed beforehand. The (polynomial) matrix is called Vandermonde matrix.

Regularization

(Linear) Ridge regression

In a case of high number of input variables we may wish to decrease the model complexity, that is the number of predictors (input variables). We could use some methods for feature selection - e.g. forward or backward selection. But we can do it also during optimization by introducing a new term.

Removing predictors from the model can be seen as settings their coefficients to zero. Instead of forcing them to be exactly zero, we can penalize them if they are too far from zero, thus enforcing them to be small in a continuous way. This way, we decrease model complexity while keeping all variables in the model. This, basically, is what ridge regression does.

(Linear) Ridge regression

The regularized version is:

$$g_{ridge}(\mathbf{w}) = \sum_{i=1}^{p}(\mathbf{w}^T\mathbf{x_i} - y_i)^2 + \lambda\sum_{i=1}^{n} w_i^2 = ||\mathbf{Xw} - \mathbf{y}||^2 + \lambda||\mathbf{w}||^2$$

where $||\mathbf{w}||^2 = \sum w_i^2$ is the squared norm of **w** or equivalently, the dot product $\mathbf{w^Tw}$, λ is scalar value determining the level of regularization. This L^2 norm regularization is often called as **ridge** regularization, thus ridge regression.

The λ parameter is the regularization penalty:

$$\lambda \to 0, \quad \mathbf{w}_{ridge} \to \mathbf{w}$$

$$\lambda \to \infty, \quad \mathbf{w}_{ridge} \to 0$$

(Linear) Ridge regression

This form of regularization still have a closed form solution:

$$\mathbf{w} = (\mathbf{X}^\mathbf{T}\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^\mathbf{T}\mathbf{y}$$

where $\mathbf{I}$ is identity matrix. If we compare this equation with equation for **w** without regularization, we see that here we add λ to the diagonal of $\mathbf{X}^\mathbf{T}\mathbf{X}$, which is know trick to improve the stability of matrix inversion.

(Linear) Lasso Regression

Lasso (Least Absolute Shrinkage and Selection Operator), is quite similar conceptually to ridge regression. It also adds a penalty for non-zero coefficients, but unlike ridge regression which penalizes sum of squared coefficients (L2 penalty), lasso penalizes the sum of their absolute values (L1 penalty). As a result, for high values of λ, many coefficients are exactly zeroed under lasso, which is never the case in ridge regression.
The cost function has this form:

$$g_{lasso}(\mathbf{w}) = \sum_{i=1}^{n} (\mathbf{w}^T \mathbf{x} - y)^2 + \lambda \sum_{i=1}^{p} |w_i| = ||\mathbf{X}\mathbf{w} - \mathbf{y}||^2 + \lambda ||\mathbf{w}||_1$$

where $||\mathbf{w}||_1 = \sum_{i=1}^{n} |w_i|$

No closed form solution. Numerical optimization.

More information about ridge and lasso regularization: https://www.youtube.com/watch?v=sO4ZirJh9ds

(Linear) Elastic Net regression

This method is a simple combination of ridge and lasso approach. Thus:

$$g_{elasticnet}(\mathbf{w}) = ||\mathbf{X}\mathbf{w} - \mathbf{y}||^2 + \lambda(\alpha||\mathbf{w}||_1 + (1-\alpha)||\mathbf{w}||^2)$$

There is an extra term for balancing L1 and L2 penalty.

Linear regression

So, which regression method to choose for your data?

In many practical cases it is just question of testing - use a cross validation, start with the basic MSE and then test the method and parameter setting for lasso and ridge.

Non-linear regression

Kernel (nonlinear) ridge regression

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{bmatrix}$$

The kernel approach can be used also here to make the regression non-linear. And, of course, kernel trick can be used.

Let's start with ridge regression. The solution still have closed form:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ . \\ . \\ y_p \end{bmatrix}$$

$$\mathbf{w} = (\mathbf{X^T X} + \lambda \mathbf{I})^{-1} \mathbf{X^T y}$$

There is some mathematical (matrix inversion or Woodbury) lemma, which allows us to rewrite this solution to:

$$\alpha = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ . \\ . \\ \alpha_p \end{bmatrix}$$

$$\mathbf{w} = \mathbf{X^T}(\mathbf{X X^T} + \lambda \mathbf{I})^{-1} \mathbf{y} = \mathbf{X^T} \alpha = \sum_{i=1}^{p} \alpha_i \mathbf{x_i}$$

where

$$\alpha = (\mathbf{X X^T} + \lambda \mathbf{I})^{-1} \mathbf{y} = (K + \lambda \mathbf{I})^{-1} \mathbf{y}$$

is an p x 1 vector of dual variables, and $\mathbf{K_{ij} = x_i^T x_j}$.

Kernel (nonlinear) ridge regression

Note that

$$\mathbf{w} = \sum_{i=1}^{p} \alpha_i \mathbf{x_i} = \mathbf{X^T} \alpha$$

is known as **dual form** of ridge regression solution. However, it is still a linear model. But, it can be easily *kernelized and thus we can use kernel trick*.

Let's go to high dimensional space. Choosing some kernel *k* with an associated feature mapping Φ, we can write

$$\mathbf{w} = \sum_{i=1}^{p} \alpha_i \phi(\mathbf{x_i}) = \phi(\mathbf{X})^T \alpha$$

Prediction for new test input **x** will be:

$$\mathbf{y} = \phi(\mathbf{x})^T \mathbf{w} = \mathbf{w^T} \phi(\mathbf{x}) = \sum_{i=1}^{p} (\alpha_i \phi(\mathbf{x_i}))^T \phi(\mathbf{x}) = \sum_{i=1}^{p} \alpha_i k(\mathbf{x_i}, \mathbf{x})$$

Kernel (nonlinear) ridge regression

Thus for our new input sample, we can effectively compute the output using some kernel function (which we choose before any computation) and we can compute the dot products between each sample from our training set and new input.

But, how to compute α_i ?

Kernel (nonlinear) ridge regression

Let's write the cost function for non-linear case:

$$g_{kernel}(\mathbf{w}) = ||\phi(\mathbf{X})\mathbf{w} - \mathbf{y}||^2 + \lambda||\mathbf{w}||^2$$

And substitute $\mathbf{w} = \phi(\mathbf{X})^T\alpha$

$$g_{kernel}(\alpha) = ||\phi(\mathbf{X})\phi(\mathbf{X})^{\mathbf{T}}\alpha - \mathbf{y}||^2 + \lambda\alpha^{\mathbf{T}}\phi(\mathbf{X})\phi(\mathbf{X})^{\mathbf{T}}\alpha$$

Let's write K instead of Φ:

$$g_{kernel}(\alpha) = ||\mathbf{K}\alpha - \mathbf{y}||^2 + \lambda\alpha^{\mathbf{T}}\mathbf{K}\alpha =$$
$$\alpha^{\mathbf{T}}\mathbf{K}\mathbf{K}\alpha - 2\mathbf{y}^{\mathbf{T}}\mathbf{K}\alpha + \mathbf{y}^{\mathbf{T}}\mathbf{y} + \lambda\alpha^{\mathbf{T}}\mathbf{K}\alpha$$

Kernel (nonlinear) ridge regression

Taking the gradient with respect to α and setting it to zero:

$$\nabla_\alpha g_{kernel}(\alpha) = 2\mathbf{K}\mathbf{K}\alpha - 2\mathbf{K}\mathbf{y} + 0 + 2\lambda\mathbf{K}\alpha$$

leads to final result:

$$\boxed{\alpha = (\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{y}}$$

$$\mathbf{K} = \phi(\mathbf{X})\phi(\mathbf{X})^T = \begin{bmatrix} \kappa(\mathbf{x_1}, \mathbf{x_1}) & \kappa(\mathbf{x_1}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_1}, \mathbf{x_p}) \\ \kappa(\mathbf{x_2}, \mathbf{x_1}) & \kappa(\mathbf{x_2}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_2}, \mathbf{x_p}) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\mathbf{x_p}, \mathbf{x_1}) & \kappa(\mathbf{x_p}, \mathbf{x_2}) & \dots & \kappa(\mathbf{x_p}, \mathbf{x_p}) \end{bmatrix}$$

Kernel	Equation
Linear	$K(x, y) = x \cdot y$
Sigmoid	$K(x, y) = \tanh(ax.y + b)$
Polynomial	$K(x, y) = (1 + x \cdot y)^d$
KMOD	$K(x, y) = a\left[\exp\left(\frac{\gamma}{\|\|x-y\|\|^2+\sigma^2}\right) - 1\right]$
RBF	$K(x, y) = \exp(-a\|\|x - y\|\|^2)$
Exponential RBF	$K(x, y) = \exp(-a\|\|x - y\|\|)$

This topic will be presented later in the Bayes theory

Logistic regression

Logistic regression

Logistic regression is a method for creating the statistical model that is used in classification problems. It allows us to take some features and predict the correct class, or better - the probability of that class.

Linear *versus* logistic regression:

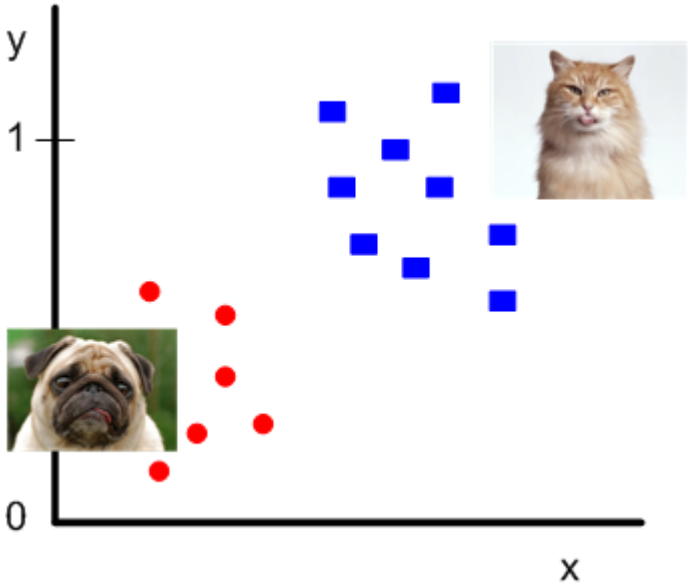
- we can predict if student pass the exam (yes/no) based on number of hours spent by studying. That is task for logistic regression.
- we can predict how many points the student get (“almost” continuous variable) based on number of hours spent by studying. That is task for linear regression.

Logistic regression

We want to predict what is the probability that we classify CAT (i.e. $y_i=1$), based on given input (vector of features $\mathbf{x_i}$) and parameters of the model $\mathbf{w}$. Written in math:

$$p(y_i=1 \mid \mathbf{x_i}, \mathbf{w}) = ?$$

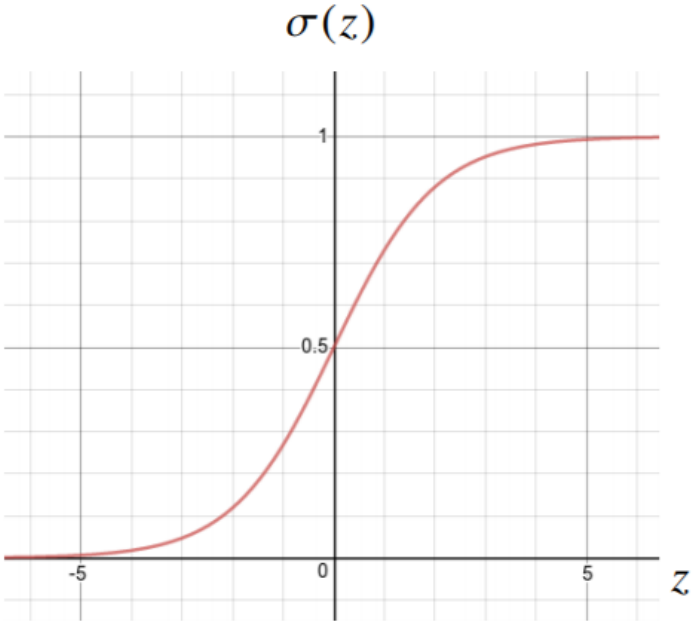
It doesn't make sense to use the linear model, because it will just fit the line through our data. But we can use some function to transform this linear model into something more convenient.



Logistic regression

For this purpose we will use the sigmoid function:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$



The useful properties of this function are:

- whatever the input is, it is transformed between 0 and 1.
- the derivative of a sigmoid with respect to input is very nice/useful (see later):

$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1-\sigma(z))$$

Logistic regression

Now, we will use this sigmoid function to transform our linear model:

$$\sigma(\mathbf{w^T x}) = \frac{1}{1+e^{-\mathbf{w^T x}}}$$

With this model, we can now write our probabilities:

$$p(y_i = 1(cat)|\mathbf{x_i}, \mathbf{w}) = \sigma(\mathbf{w^T x_i}) = \frac{1}{1+e^{-\mathbf{w^T x_i}}}$$

$$p(y_i = 0(dog)|\mathbf{x_i}, \mathbf{w}) = 1 - \sigma(\mathbf{w^T x_i}) = 1 - \frac{1}{1+e^{-\mathbf{w^T x_i}}}$$

Now we combine these two equation into a single one:

$$p(y_i|x_i, \mathbf{w}) = \left[\sigma(\mathbf{w^T x_i})\right]^{y_i} . \left[1 - \sigma(\mathbf{w^T x_i})\right]^{(1-y_i)}$$

Logistic regression

The last equation is Bernoulli distribution, which is generally described as:

$$P\left[X = 1\right] = 1 - P\left[X = 0\right] = p.$$

We can now look on our data in a different way: We can interpret each label as a Bernoulli random variable given the model described by sigma function.

Now, we define “likelihood” as a probability that observed data (y) come from specific model (Bernoulli) with parameters **w**. This likelihood function for my independent and identically distributed random variable has the following form:

$$L(\mathbf{w};\mathbf{y},\mathbf{X}) = \prod_i P(y_i|\mathbf{w},\mathbf{x_i}) = \prod_i \sigma(\mathbf{w^T x_i})^{y_i} \cdot \left[1 - \sigma(\mathbf{w^T x_i})\right]^{(1-y_i)}$$

Logistic regression

In this function we just multiply the probabilities - we can do this because we are assuming that our datapoints (measurements) are independent, which is in most cases a reasonable assumption (but not always!).

For what is this likelihood function good for?

My aim is to maximize this function (given a measured data) and I will maximize it by optimization of model parameters **w**.

Therefore a maximum likelihood approach is used:

$$\max_{\mathbf{w}} \quad L(\mathbf{w}; \mathbf{y}, \mathbf{X})$$

Logistic regression

It turns out that maximizing likelihood function can be difficult due to multiplications. Better way is to maximize Log of likelihood function:

$$LogL(\mathbf{w};\mathbf{y},\mathbf{X}) = \log L(\mathbf{w};\mathbf{y},\mathbf{X}) =$$
$$\log(\textstyle\prod_i \sigma(\mathbf{w^T x_i})^{y_i} \,.\, \left[1-\sigma(\mathbf{w^T x_i})\right]^{(1-y_i)}) =$$
$$\textstyle\sum_i y_i \log(\sigma(\mathbf{w^T x_i}) + (1-y_i)\log(1-\sigma(\mathbf{w^T x_i}))$$

We have to maximize this function with respect to **w**. So we must take the derivative of *LogL*.

Logistic regression

We will skip this derivation and we write the final expression for derivative:

$$\frac{\partial LogL(\mathbf{w;y,X})}{\partial w^{(j)}} = \sum_i [y_i - \sigma(\mathbf{w^T x_i})] x_i^{(j)}$$

This is derivative with respect to each element *j* of vector ***w***, denoted as $w^{(j)}$.

And finally we can write the gradient ascent optimization (we will go “uphill”):

$$w_{new}^{(j)} = w_{old}^{(j)} + \nu \sum_i [y_i - \sigma(\mathbf{w^T x_i})] x_i^{(j)}$$

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \nu \sum_i [y_i - \sigma(\mathbf{w^T x_i})] \mathbf{x}_i$$

new = old + step * gradient

Logistic regression

In logistic regression no assumptions are made about the distributions of the input variables. However, the input variables should not be highly correlated with one another because this could cause problems with estimation.

Large sample sizes are required for logistic regression to provide sufficient numbers in both categories of the response variable. The more input variables, the larger the sample size required (remember the Curse of dimensionality).